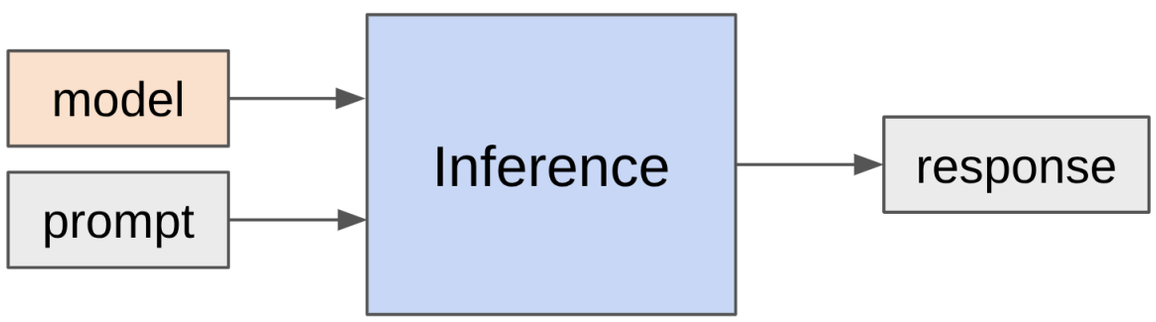
Lecture 10: inference

> 第 10 讲：推理



Understanding the inference workload

> 理解推理工作负载

Taking shortcuts (lossy)

> 走捷径（有损）

Summary: reduce inference complexity without hurting accuracy

> 总结：在不损害精度的前提下降低推理复杂度

From scratch recipe:

> 从零训练的方案：

1. Define faster model architecture

> 定义更快的模型架构

2. Train faster model

> 训练更快的模型

Distillation recipe:

> 蒸馏的方案：

1. Define faster model architecture

> 定义更快的模型架构

2. Initialize weights using original model (which has a different architecture)

> 用原始模型（架构不同）来初始化权重

3. Repair faster model (distillation)

> 修复更快的模型（蒸馏）

Use shortcuts but double check (lossless)

> 走捷径但加以校验（无损）

Handling dynamic workloads

> 处理动态工作负载

Batching over sequences in live traffic is tricky because:

> 对线上流量中的序列做批处理很棘手，原因在于：

1. Requests arrive at different times (waiting for batch is bad for early requests)

> 请求到达时间不同（等待凑批对早到的请求不利）

2. Sequences have shared prefixes (e.g., system prompts, generating multiple samples)

> 序列有共享前缀（例如系统提示词、生成多个样本）

3. Sequences have different lengths (padding is inefficient)

> 序列长度不同（填充很低效）

Summary

> 总结

- Inference is important (actual use, evaluation, reinforcement learning)

> 推理很重要（实际使用、评测、强化学习）

- Different characteristics compared to training (memory-bound, dynamic)

> 与训练相比特性不同（访存受限、动态）

- Techniques: new architectures, quantization, pruning/distillation, speculative sampling

> 技术：新架构、量化、剪枝/蒸馏、投机采样

- Ideas from systems (speculative execution, paging)

> 来自系统的思想（投机执行、分页）

- New architectures have huge potential for improvement

> 新架构有巨大的改进潜力

Inference shows up in many places:

> 推理出现在很多场景：

- Actual use (chatbots, code completion, agents, batch data processing)

> 实际使用（聊天机器人、代码补全、agent、批量数据处理）

- Model evaluation (e.g., on instruction following)

> 模型评测（例如指令遵循）

- Reinforcement learning (sample many generations, then apply score)

> 强化学习（采样大量生成结果，再打分）

Why **efficiency** matters: training is one-time cost, inference is repeated many times

> 为什么效率重要：训练是一次性成本，推理要重复很多次

- OpenAI processes ~8.6T tokens per day [article]

> OpenAI 每天处理约 8.6T 个 token [article]

- For reference, DeepSeek v4 was trained on 32T tokens [DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]

> 作为参照，DeepSeek v4 的训练用了 32T 个 token [DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]

Moreover:

> 此外：

- Chatbots: most tokens are meant for human consumption (humans are bottleneck)

> 聊天机器人：大多数 token 是给人看的（人是瓶颈）

- Agents: query → internal trace → output for human (number of tokens generated can grow unbounded)

> Agent：查询 → 内部轨迹 → 给人的输出（生成的 token 数量可以无限制增长）

- Tokens generated = compute spent

> 生成的 token 数 = 消耗的算力

Companies doing inference (a big deal for anyone who has a product or platform):

> 做推理的公司（对任何有产品或平台的人来说都是大事）：

- Providers serving closed models (OpenAI, Anthropic, Google, etc.)

> 服务闭源模型的提供商（OpenAI、Anthropic、Google 等）

- Providers serving open-weight models (Together, Fireworks, Baseten, DeepInfra, Groq, Cerebras, etc.)

> 服务开放权重模型的提供商（Together、Fireworks、Baseten、DeepInfra、Groq、Cerebras 等）

Open-source packages:

> 开源软件包：

- vLLM: from Berkeley, pioneered PagedAttention, popular and good default [GitHub]

> vLLM：来自 Berkeley，首创 PagedAttention，流行且是不错的默认选择 [GitHub]

- SGLang: from Berkeley, pioneered RadixAttention, good for agentic workloads [project]

> SGLang：来自 Berkeley，首创 RadixAttention，适合 agentic 负载 [project]

- TensorRT-LLM: from NVIDIA, highly optimized for GPUs [article]

> TensorRT-LLM：来自 NVIDIA，针对 GPU 高度优化 [article]

- llama.cpp: C++ only, supports CPU inference, runs locally [GitHub]

> llama.cpp：纯 C++，支持 CPU 推理，可在本地运行 [GitHub]

Inference is huge. Important to make it fast.

> 推理规模巨大，让它变快很重要。

What does "fast" mean (metrics)?

> “快”意味着什么（指标）？

- Time-to-first-token (TTFT): how long user waits before any generation happens (for interactive applications)

> 首 token 时间（TTFT）：在任何生成发生之前用户要等多久（面向交互式应用）

- Latency (seconds/token): how fast tokens appear for *one* query (for interactive applications)

> 时延（秒/token）：单条查询的 token 出现得多快（面向交互式应用）

- Throughput (tokens/second): how fast tokens appear for *many* queries (for batch processing)

> 吞吐（tokens/秒）：大量查询下 token 出现得多快（面向批处理）

What governs efficiency?

> 什么决定效率?

- Training (supervised): you see all tokens, can parallelize over sequence (matmul in Transformer)

> 训练（有监督）：能看到所有 token，可以在序列维度上并行（Transformer 中的矩阵乘法）

- Inference: you have to generate sequentially, can't parallelize over generation, so harder to fully utilize compute

> 推理：必须按顺序生成，无法在生成维度上并行，所以更难充分利用算力

[Scaling book chapter on Transformers]

> [Scaling book 中关于 Transformer 的章节]

Notation (similar to einops):

> 记号（类似 einops）：

- Symbols denote dimensions (and their length): B (batch), T (sequence), D (model dim), H (head dim)

> 符号表示维度（及其长度）：B（batch）、T（序列）、D（模型维度）、H（头维度）

- Example: BTD x DH → BTH

> 示例：BTD x DH → BTH

- Contracting (red) dimensions appear in both operands and disappear from result

> 收缩维（红色）出现在两个操作数中，并从结果中消失

- Regular (black) dimensions appear in one operand and stay in result

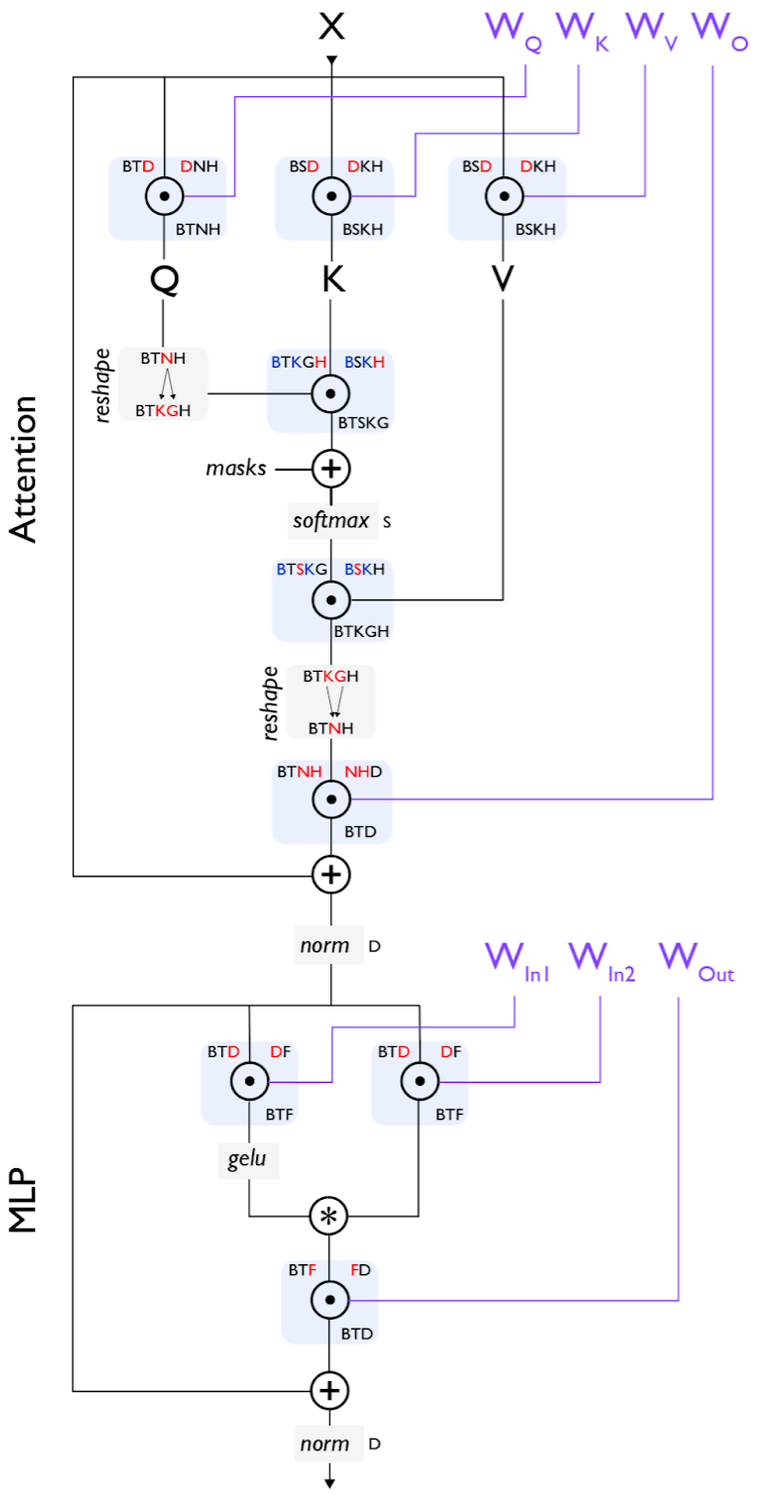
> 常规维（黑色）只出现在一个操作数中，并保留在结果里

- Example: BD x BD → B

示例：BD x BD → B

- Batching (blue) dimensions appear in both operands and stay in result

批处理维（蓝色）出现在两个操作数中，并保留在结果里



symbol	dimension
B	batch
L	number of layers
T	sequence length (query)
S	sequence length (key value)
V	vocab
D	d_model, embedding dimension
F	MLP hidden dimension
H	attention head dimension
N	number of query heads
K	number of key/value heads
G	q heads per kv head = N // K

Conventions:

约定：

- F = 4 D (MLP up-projects into 4x the model dimension)

F = 4 D（MLP 向上投影到 4 倍模型维度）

- D = N H (model dimension split across N heads)

D = N H（模型维度拆分到 N 个头）

- N = K G (for GQA, number of heads split across K groups)

N = K G（对 GQA 而言，头数拆分到 K 个组）

- S = T (during training, condition on S input tokens to predict T output tokens)

S = T（训练时，以 S 个输入 token 为条件预测 T 个输出 token）

Setup: multiply X (B x D) and W (D x F) matrix

设定：矩阵 X (B x D) 与 W (D x F) 相乘

Intuition: B is batch size, D is hidden dimension, F is up-projection dimension in MLP

直觉：B 是 batch 大小，D 是隐藏维度，F 是 MLP 的向上投影维度

Let's do FLOPs and memory read/write accounting for the matrix multiplication (X * W).

> 我们来为矩阵乘法 (X * W) 做一次 FLOPs 和内存读写的核算。

1. Read X (B x D) from HBM

> 从 HBM 读 X (B x D)

2. Read W (D x F) from HBM

> 从 HBM 读 W (D x F)

3. Compute Y = X (B x D) @ W (D x F)

> 计算 Y = X (B x D) @ W (D x F)

4. Write Y (B x F) to HBM

> 向 HBM 写 Y (B x F)

Recall that **arithmetic intensity** is how much compute we do per byte transferred (want to be high).

> 回顾一下：算术强度（arithmetic intensity）是每传输一个字节所做的计算量（希望它高）。

Assuming B is much less than D and F, then we can simplify:

> 假设 B 远小于 D 和 F，则可以化简：

Accelerator intensity of H100:

> H100 的加速器强度：

If computation intensity > accelerator intensity, **compute-bound** (good)

> 若计算强度 > 加速器强度，则算力受限（好）

If computation intensity < accelerator intensity, **memory-bound** (bad)

> 若计算强度 < 加速器强度，则访存受限（坏）

Conclusion: compute-bound iff B > 295

> 结论：当且仅当 B > 295 时为算力受限

Extreme case (B = 1, corresponding to matrix-vector product):

> 极端情形（B = 1，对应矩阵-向量乘法）：

- Arithmetic intensity: 1

> 算术强度：1

- Memory-bound (read D x F matrix, perform only 2 D F FLOPs)

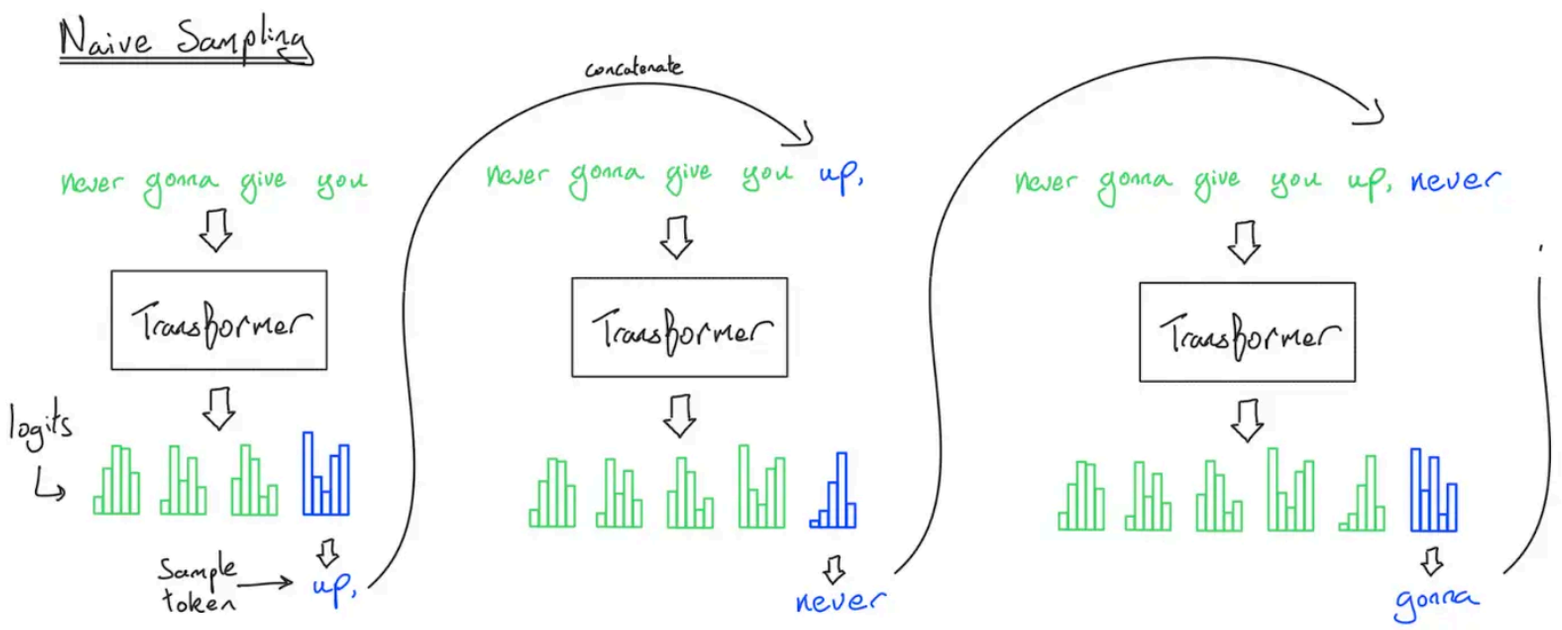
> 访存受限（读取 D x F 的矩阵，却只做 2 D F FLOPs）

- This is basically what happens with inference...

> 推理的情形基本就是如此……

[Scaling book chapter on inference]

> [Scaling book 中关于推理的章节]



Naive inference: to generate each token, feed history into Transformer

> 朴素推理：每生成一个 token，都要把全部历史喂给 Transformer

Complexity: generating T tokens requires O(T^3) FLOPs (one feedforward pass is O(T^2))

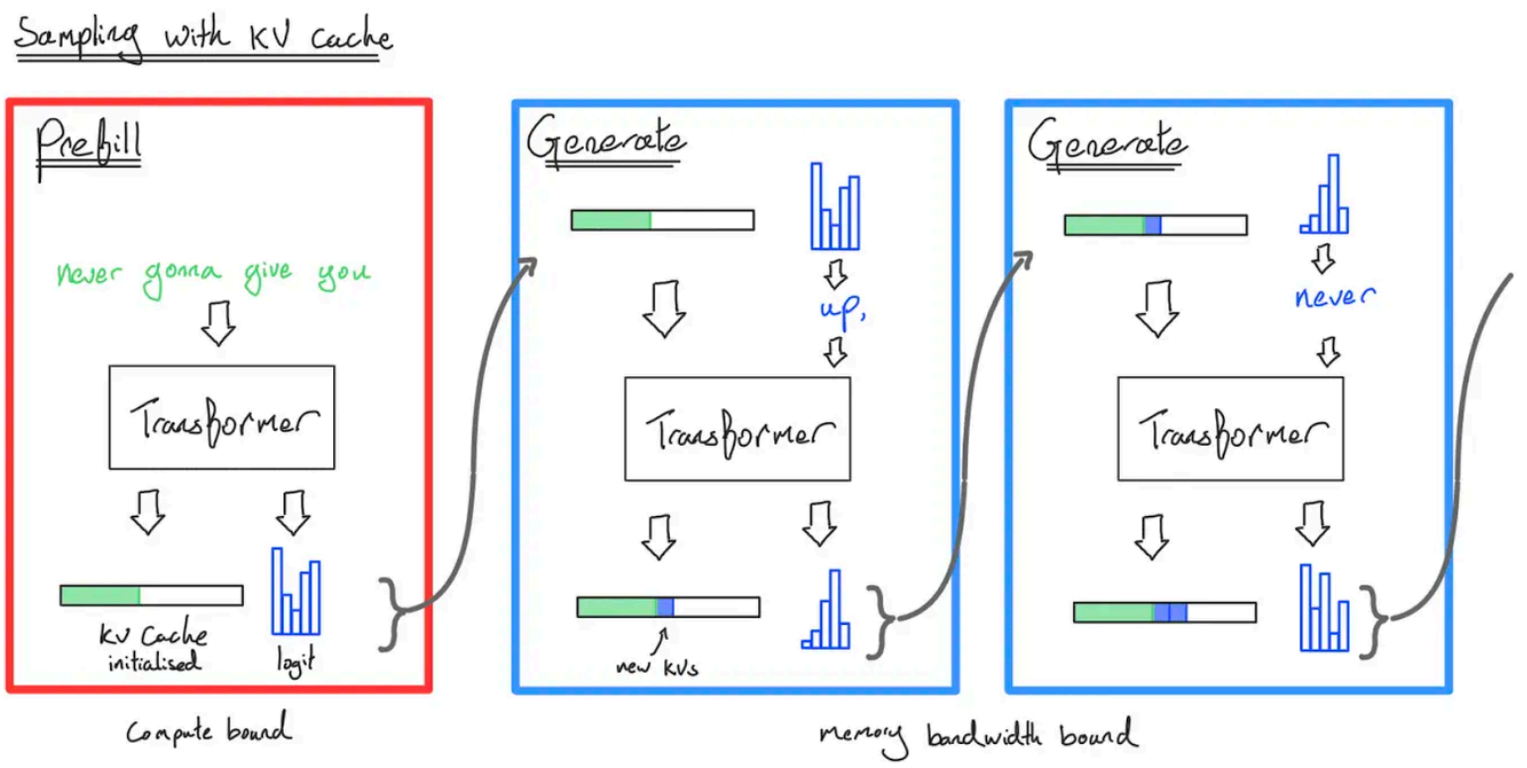
> 复杂度：生成 T 个 token 需要 O(T^3) FLOPs（一次前向传播是 O(T^2)）

Observation: a lot of the work can be shared across prefixes

> 观察：大量计算可以在不同前缀之间共享

Solution: store **KV cache** in HBM

> 解法：把 KV cache 存到 HBM 里



KV cache: for every sequence (B), token (S), layer (L), head (K), store an H-dimensional vector

> KV cache：对每个序列 (B)、每个 token (S)、每层 (L)、每个头 (K)，存一个 H 维向量

Two stages of inference:

> 推理的两个阶段：

1. **Prefill**: given a prompt, encode into vectors (parallelizable like in training)

> Prefill：给定提示词，编码成向量（可以像训练那样并行）

2. **Generation**: generate new response tokens (sequential)

> 生成：生成新的回复 token（串行）

Let's compute the FLOPs and memory IO for both the MLP and attention layers.

> 我们来算一下 MLP 层和注意力层的 FLOPs 与内存 IO。

S is the number of tokens we're conditioning on, T is the number of tokens we're generating.

> S 是作为条件（已知）的 token 数，T 是要生成的 token 数。

Later, we'll specialize to prefill (T = S) and generation (T = 1).

> 之后我们会特化到 prefill（T = S）和生成（T = 1）两种情形。

MLP layers (only looking at the matrix multiplications)

> MLP 层（只看矩阵乘法）

1. Read X (B x T x D) from HBM

> 从 HBM 读 X (B x T x D)

2. Read Wup (D x F), Wgate (D x F), Wdown (F x D) from HBM

> 从 HBM 读 Wup (D x F)、Wgate (D x F)、Wdown (F x D)

3. Compute U = X (B x T x D) @ Wup (D x F)

> 计算 U = X (B x T x D) @ Wup (D x F)

4. Write U (B x T x F) to HBM

> 向 HBM 写 U (B x T x F)

5. Compute G = X (B x T x D) @ Wgate (D x F)

> 计算 G = X (B x T x D) @ Wgate (D x F)

6. Write G (B x T x F) to HBM

> 向 HBM 写 G (B x T x F)

7. Compute Y = GeLU(G)*U (B x T x F) @ Wdown (F x D)

> 计算 Y = GeLU(G)*U (B x T x F) @ Wdown (F x D)

8. Write Y (B x T x D) to HBM

> 向 HBM 写 Y (B x T x D)

Assume that B*T is much smaller than D and F.

> 假设 B*T 远小于 D 和 F。

For the two stages:

> 对两个阶段：

1. Prefill: easy to make compute-bound (good) by making `B*T` large enough (large batches, long sequences)

> Prefill：只要让 B*T 足够大（大批量、长序列），就很容易变成算力受限（好）

2. Generation: two problems

> 生成：两个问题

- Generating one token at a time (T = 1)

> 一次只生成一个 token（T = 1）

- B is number of concurrent requests, unpredictable for interactive applications

> B 是并发请求数，对交互式应用来说不可预测

Attention layers (focusing on the matrix multiplications with FlashAttention)

> 注意力层（聚焦 FlashAttention 下的矩阵乘法）

- S is number of previous tokens (already generated)

> S 是之前的 token 数（已生成）

- T is number of next tokens (to generate logits for)

> T 是接下来的 token 数（要为其生成 logits）

1. Read Q (B x T x D), K (B x S x D), V (B x S x D) from HBM

> 从 HBM 读 Q (B x T x D)、K (B x S x D)、V (B x S x D)

2. Compute A = Q (B x T x D) @ K (B x S x D)

> 计算 A = Q (B x T x D) @ K (B x S x D)

3. Compute Y = softmax(A) (B x S x T x K x G) @ V (B x S x K x H)

> 计算 Y = softmax(A) (B x S x T x K x G) @ V (B x S x K x H)

4. Write Y (B x T x D) to HBM

> 向 HBM 写 Y (B x T x D)

For the two stages:

> 对两个阶段：

1. Prefill: T = S

> Prefill：T = S

2. Generation: T = 1

> 生成：T = 1

Unlike MLPs, no dependence on B, so batching doesn't help!

> 与 MLP 不同，这里不依赖 B，所以批处理帮不上忙！

Why?

> 为什么?

- In MLP layers, every sequence hits the same MLP weights (Wup, Wgate, Wdown don't depend on B)

> MLP 层里，每条序列用的都是同一组 MLP 权重（Wup、Wgate、Wdown 不依赖 B）

- In attention layers, every sequence has its own KV cache vectors (Q, K, V all depend on B)

> 注意力层里，每条序列有自己的 KV cache 向量（Q、K、V 都依赖 B）

Summary:

> 总结：

- Prefill is compute-bound, generation is memory-bound

> Prefill 是算力受限，生成是访存受限

- Prefill MLP intensity: `B*S`

> Prefill 的 MLP 强度：B*S

- Prefill attention intensity: `S/2`

> Prefill 的注意力强度：S/2

- Generation MLP intensity: `B` (requires concurrent requests)

> 生成的 MLP 强度：B（需要并发请求）

- Generation attention intensity: `<1` (impossible to improve)

> 生成的注意力强度：<1（不可能改进）

So we have shown that inference is memory-bound.

> 所以我们已经证明了推理是访存受限的。

Let us now compute the theoretical maximum latency and throughput of a single request.

> 现在来计算单个请求在理论上的最大时延和吞吐。

Assumption: can overlap compute and communication perfectly and ignore overhead.

> 假设：计算与通信可以完美重叠，且忽略开销。

Instantiate latency and throughput for Llama 2 13B on an H100:

> 代入 Llama 2 13B 在一块 H100 上的具体数值来算时延和吞吐：

Result: worse latency, better throughput

> 结果：时延变差，吞吐变好

Result: even worse latency, even better throughput

> 结果：时延更差，吞吐更好

Result: doesn't fit into memory and throughput gains are diminishing too...

> 结果：内存放不下，而且吞吐的收益也在递减……

What increasing batch size does:

> 增大 batch 大小会带来什么：

- Worsens latency because larger KV cache (O(B) size) to read/write

> 时延变差，因为要读写的 KV cache 更大（O(B) 规模）

- Improves throughput because amortizes the cost of reading parameters

> 吞吐变好，因为读取参数的成本被摊薄了

Tradeoff between latency and throughput:

> 时延与吞吐的权衡：

1. Smaller batch sizes yield better latency but worse throughput

> 更小的 batch 带来更好的时延，但吞吐更差

2. Larger batch sizes yield better throughput but worse latency

> 更大的 batch 带来更好的吞吐，但时延更差

Easy parallelism: if you launch M copies of the model, latency is the same, throughput increases by M!

> 简单的并行：如果部署 M 份模型副本，时延不变，吞吐提升 M 倍！

Harder parallelism: shard the model and the KV cache [Scaling book chapter on inference]

> 更难的并行：对模型和 KV cache 做分片 [Scaling book 中关于推理的章节]

Note: time-to-first-token (TTFT) is essentially a function of prefill time

> 注：首 token 时间（TTFT）基本上是 prefill 时间的函数

Use smaller batch sizes during prefill for faster TTFT

> prefill 阶段用更小的 batch，以获得更快的 TTFT

Use larger batch sizes during generation to improve throughput

> 生成阶段用更大的 batch 来提升吞吐

Recall that memory is the bottleneck for inference.

> 回顾一下：内存是推理的瓶颈。

So let's try to reduce the size of the KV cache

> 所以让我们设法缩小 KV cache 的规模

...but make sure we don't lose too much accuracy.

> ……但别损失太多精度。

Grouped-query attention (GQA)

[Ainslie+ 2023]

> 分组查询注意力（GQA） [Ainslie+ 2023]

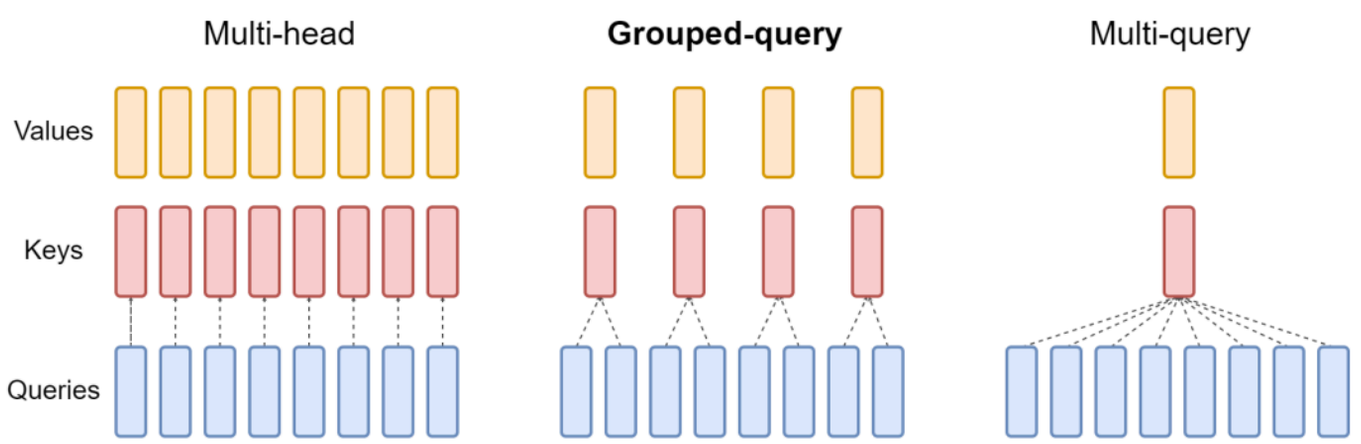


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Idea: N query heads, but only K key and value heads, each interacting with N/K query heads

> 想法：N 个查询头，但只有 K 个 key 头和 value 头，每个与 N/K 个查询头交互

Multi-headed attention (MHA): K=N

> 多头注意力（MHA）：K=N

Multi-query attention (MQA): K=1

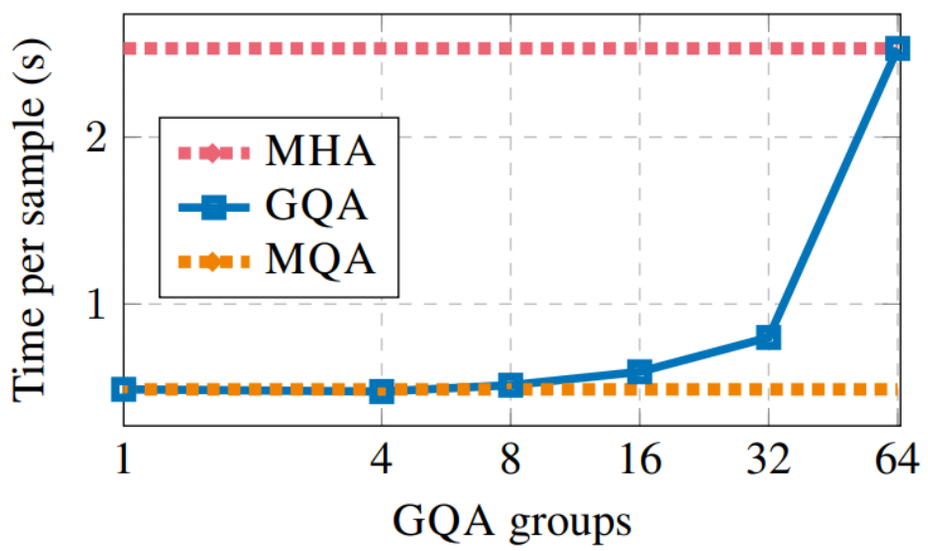
> 多查询注意力（MQA）：K=1

Group-query attention (GQA): K is somewhere in between

> 分组查询注意力（GQA）：K 介于两者之间

Latency/throughput improves: [Ainslie+ 2023]

> 时延/吞吐的改善： [Ainslie+ 2023]



Why does GQA improve latency and throughput?

> 为什么 GQA 能改善时延和吞吐?

GQA reduces the KV cache by a factor of N/K.

> GQA 把 KV cache 缩小 N/K 倍。

Reminder: reducing memory usage leads to speedup (since we're memory-bound).

> 提醒：减少内存用量会带来加速（因为我们是访存受限的）。

Result: Worse latency, but better throughput (and it fits in memory now!)

> 结果：时延更差，但吞吐更好（而且现在内存放得下了！）

Result: Worse latency, but better throughput (and still fits in memory!)

> 结果：时延更差，但吞吐更好（而且内存仍然放得下！）

Check that accuracy doesn't drop: [Ainslie+ 2023]

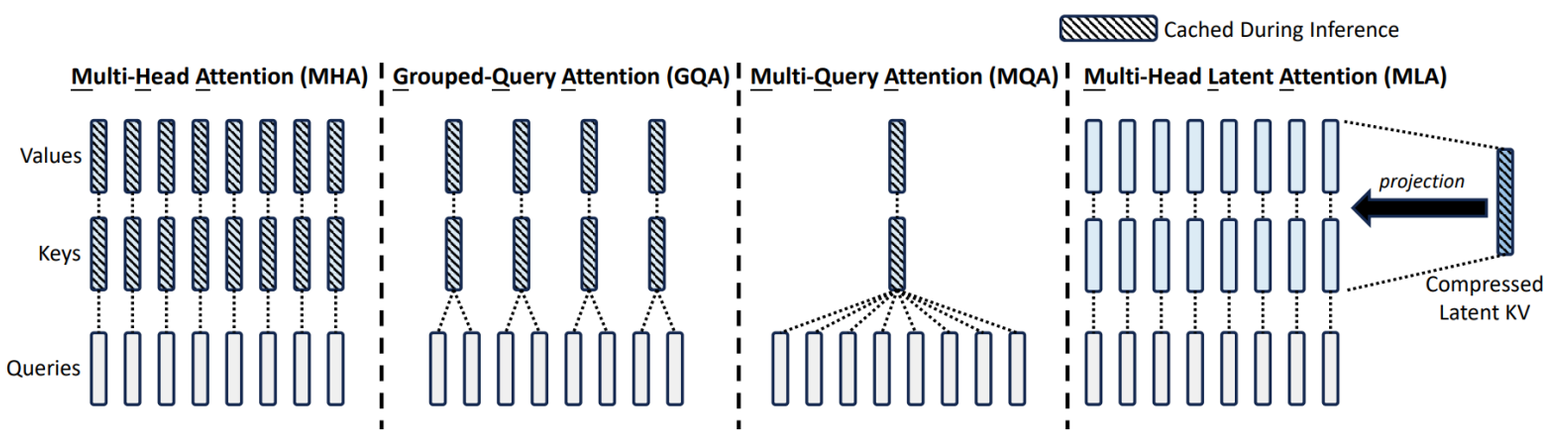
> 确认精度没有下降： [Ainslie+ 2023]

Model	T_{infer}	Average	CNN	arXiv	PubMed	MediaSum	MultiNews	WMT	TriviaQA
	s		R_1	R_1	R_1	R_1	R_1	BLEU	F1
MHA-Large	0.37	46.0	42.9	44.6	46.2	35.5	46.6	27.7	78.2
MHA-XXL	1.51	47.2	43.8	45.6	47.5	36.4	46.9	28.4	81.9
MQA-XXL	0.24	46.6	43.0	45.0	46.9	36.1	46.5	28.5	81.3
GQA-8-XXL	0.28	47.1	43.5	45.4	47.7	36.3	47.2	28.4	81.6

Multi-head latent attention (MLA)

[DeepSeek-AI+ 2024]

> 多头潜在注意力（MLA） [DeepSeek-AI+ 2024]



Normal attention: KV cache consists of K = W_K h, V = W_V h (N*H dimensions)

> 普通注意力：KV cache 由 K = W_K h、V = W_V h 组成（N*H 维）

MLA: store compressed vector c = W_c h (C dimensions), project up to K = W_K c, V = W_V c when needed

> MLA：存压缩向量 c = W_c h（C 维），需要时再向上投影出 K = W_K c、V = W_V c

DeepSeek v2: reduce N*H = 16384 to C = 512

> DeepSeek v2：把 N*H = 16384 降到 C = 512

Wrinkle: MLA is not compatible with RoPE, so need to add additional 64 dimensions for RoPE, so 512 + 64 = 576 total dimensions

> 小麻烦：MLA 与 RoPE 不兼容，所以需要额外加 64 维给 RoPE，总共就是 512 + 64 = 576 维

Latency/throughput improvements follow similarly from the KV cache reduction as argued earlier

> 按照前面的论证，KV cache 的缩小同样会带来时延/吞吐的改善

Let's now check the accuracy.

> 现在来检查精度。

First, MHA is better than GQA (though more expensive) [Table 8] [DeepSeek-AI+ 2024]

> 第一，MHA 比 GQA 好（虽然更贵）[Table 8] [DeepSeek-AI+ 2024]

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	**37.0**
MMLU (Acc.)	5-shot	37.9	41.2	**45.2**
C-Eval (Acc.)	5-shot	30.0	37.7	**42.9**
CMMLU (Acc.)	5-shot	34.6	38.4	**43.5**

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Second, MLA is even a bit better than MHA (and much cheaper) [Table 9] [DeepSeek-AI+ 2024]

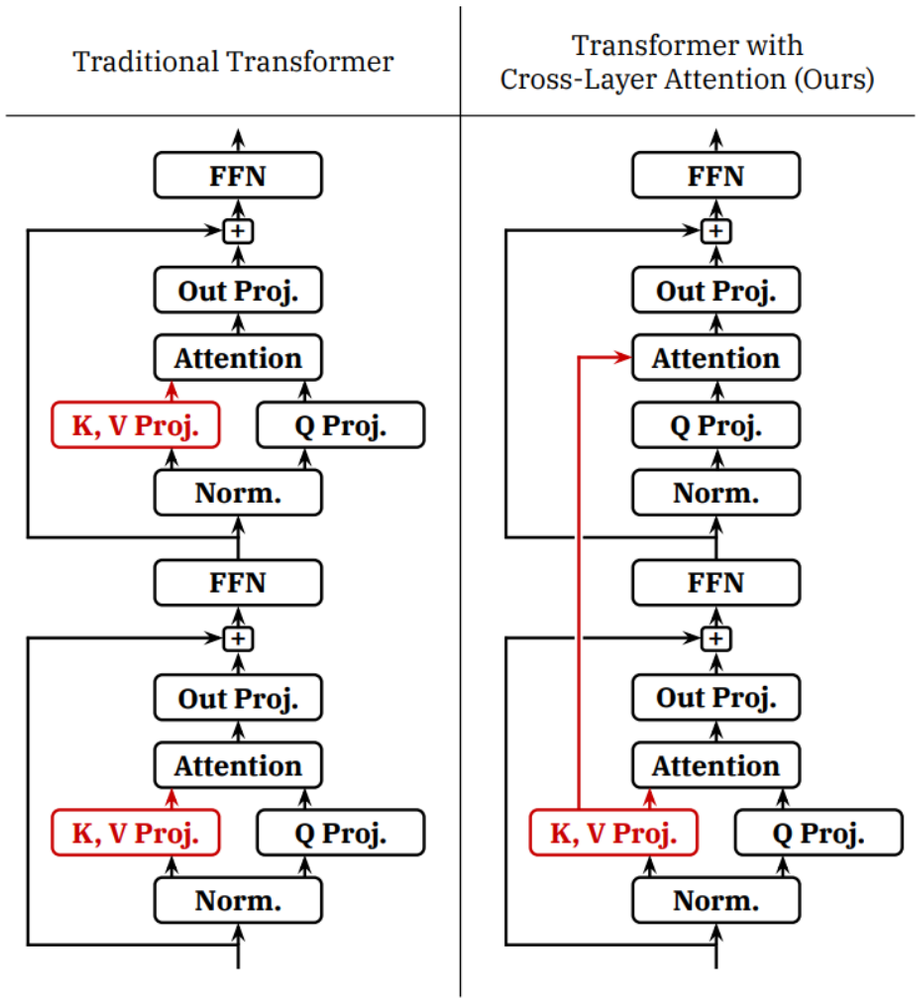
> 第二，MLA 甚至比 MHA 还好一点（而且便宜得多）[Table 9] [DeepSeek-AI+ 2024]

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	**39.0**	46.6	**50.7**
MMLU (Acc.)	5-shot	48.7	**50.0**	57.5	**59.0**
C-Eval (Acc.)	5-shot	**51.6**	50.9	57.9	**59.2**
CMMLU (Acc.)	5-shot	52.3	**53.4**	60.7	**62.5**

Cross-layer attention (CLA)

[Brandon+ 2024]

> 跨层注意力（CLA） [Brandon+ 2024]

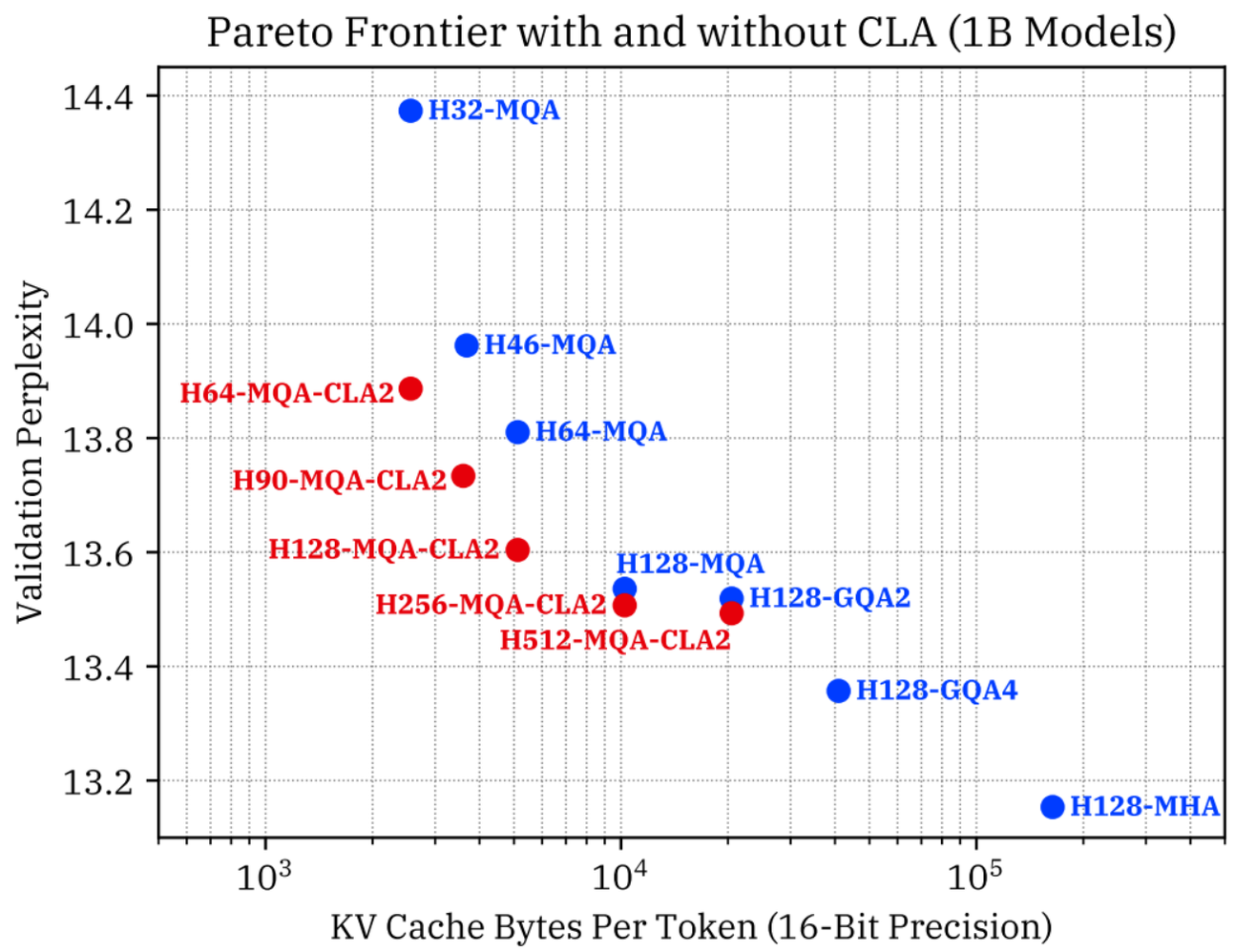


Idea: share KVs across **layers** (just as GQA shares KVs across heads)

> 想法：跨层共享 KV（就像 GQA 跨头共享 KV）

Empirically improves the pareto frontier of accuracy and KV cache size (latency and throughput)

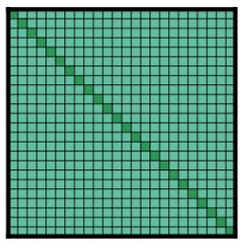
> 经验上改善了精度与 KV cache 大小（时延与吞吐）之间的帕累托前沿



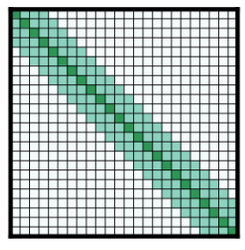
Local (sliding window) attention

[Beltagy+ 2020][Child+ 2019][Jiang+ 2023]

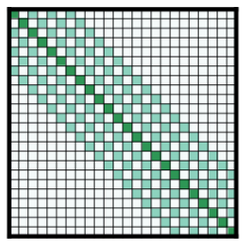
> 局部（滑动窗口）注意力 [Beltagy+ 2020][Child+ 2019][Jiang+ 2023]



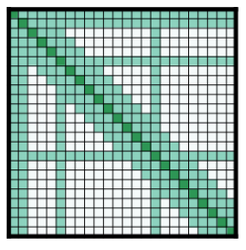
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window

Idea: just look at the local context, which is most relevant for modeling

> 想法：只看局部上下文，它对建模最相关

Effective context scales linearly with the number of layers

> 有效上下文随层数线性增长

KV cache is independent of sequence length!

> KV cache 与序列长度无关！

Problem: this can still hurt accuracy

> 问题：这仍可能损害精度

Solution: interleave local attention with global attention (hybrid layers)

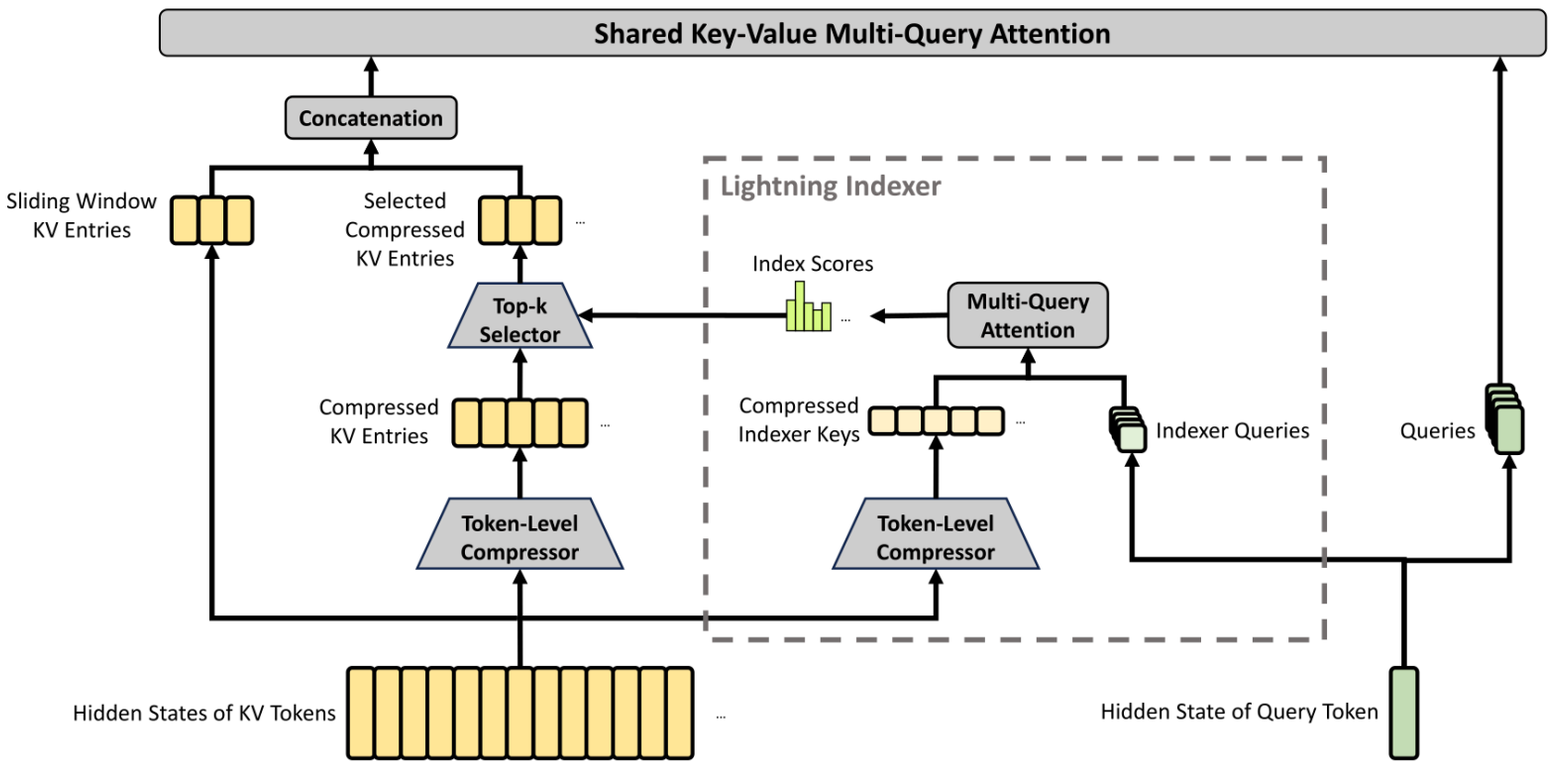
> 解法：让局部注意力与全局注意力交错（混合层）

DeepSeek v4 attention

> DeepSeek v4 注意力

- Supports 1M context length [DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]

> 支持 1M 上下文长度 [DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]



- Compressed Sparse Attention (CSA): compresses every m tokens into 1

> Compressed Sparse Attention（CSA）：把每 m 个 token 压缩成 1 个

- DeepSeek Sparse Attention (DSA): selects the top k

> DeepSeek Sparse Attention（DSA）：选出 top k

- Heavily Compressed Attention (HCA): compresses even more

> Heavily Compressed Attention（HCA）：压缩得更狠

Summary:

> 总结：

- Goal: reduce the KV cache size (since inference is memory-bound) without hurting accuracy

> 目标：在不损害精度的前提下缩小 KV cache（因为推理是访存受限的）

- Lower-dimensional KV cache (GQA, MLA, CLA)

> 更低维的 KV cache（GQA、MLA、CLA）

- Local attention (truncates the KV cache) on some of the layers

> 在部分层上使用局部注意力（截断 KV cache）

- Other ideas: linear attention / state-space-models (Mamba 2, GatedDeltaNet), diffusion models

> 其他思路：线性注意力 / 状态空间模型（Mamba 2、GatedDeltaNet）、扩散模型

Key idea: reduce the precision of numbers

> 核心想法：降低数值的精度

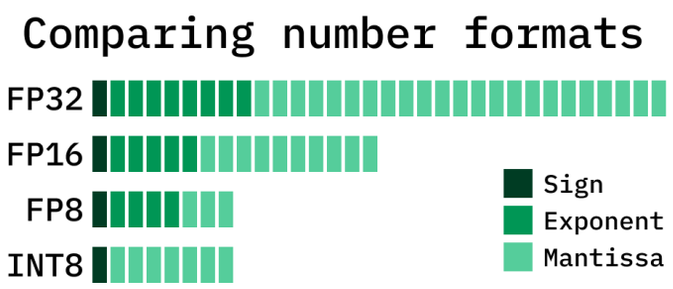
Less memory means higher latency/throughput (since inference is memory-bound).

> 内存更少意味着更好的时延/吞吐（因为推理是访存受限的）。

Of course we have to worry about accuracy...

> 当然，我们还得担心精度……

[article]



- fp32 (4 bytes): needed for parameters and optimizer states during training

> fp32（4 字节）：训练时参数和优化器状态需要它

- bf16 (2 bytes): default for inference

> bf16（2 字节）：推理的默认选择

- fp8 (1 byte) [-240, 240] for e4m3 on H100s: can train if you dare [Peng+ 2023]

> fp8（1 字节）H100 上 e4m3 的范围是 [-240, 240]：胆子大也可以拿来训练 [Peng+ 2023]

- int8 (1 byte) [-128, 127]: less accurate but cheaper than fp8, but for inference only [Baalen+ 2023]

> int8（1 字节）[-128, 127]：精度不如 fp8 但更便宜，不过仅用于推理 [Baalen+ 2023]

- int4 (0.5 bytes) [-8, 7]: cheaper, even less accurate [Baalen+ 2023]

> int4（0.5 字节）[-8, 7]：更便宜，精度也更低 [Baalen+ 2023]

[Overview of approaches]

> [方法概览]

Quantization-aware training (QAT)

> 量化感知训练（QAT）

- During training, quantize-and-dequantize during the forward pass to simulate quantization errors

> 训练期间在前向传播中做量化-反量化，以模拟量化误差

- Pro: weights are trained to work with quantization

> 优点：权重在训练中学会了适应量化

- Con: requires expensive large-scale training

> 缺点：需要昂贵的大规模训练

Post-training quantization (PTQ):

> 训练后量化（PTQ）：

- Done after training, so much cheaper

> 在训练之后进行，因此便宜得多

- Run on sample data to determine scale and zero point for each layer or tensor

> 在采样数据上运行，为每层或每个张量确定 scale 和 zero point

- GPTQ: use Hessian information to update non-quantized weights to account for quantization error

[Frantar+ 2022]

> GPTQ：利用 Hessian 信息更新尚未量化的权重，以补偿量化误差 [Frantar+ 2022]

Activation-aware quantization (AWQ)

> 激活感知量化（AWQ）

[Lin+ 2023]

- Observation: some activation channels are large

> 观察：某些激活通道的值很大

- Weights that hit those matter more

> 与这些通道相乘的权重更重要

- Allocate more precision to those weights

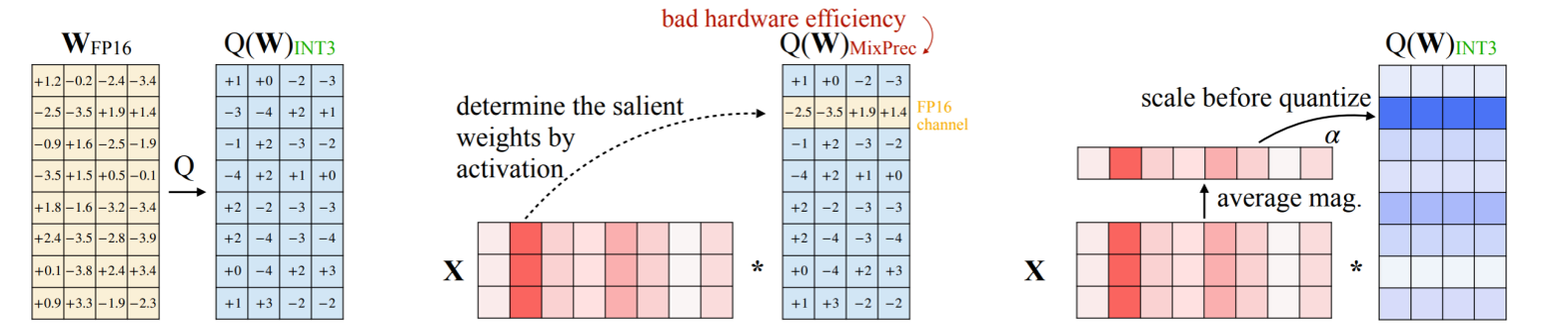
> 给这些权重分配更高的精度

- Idea: select which weights (0.1-1%) to keep in high precision based on activations

> 想法：根据激活值选出哪些权重（0.1-1%）保留高精度

- fp16 → int3 produces 4x lower memory, 3.2x speedup

> fp16 → int3 带来 4 倍的内存降低、3.2 倍的加速



(a) RTN quantization (**PPL 43.2**) (b) Keep 1% salient weights in FP16 (**PPL 13.0**) (c) Scale the weights before quantization (**PPL 13.0**)

Key idea: just rip out parts of an expensive model to make it cheaper

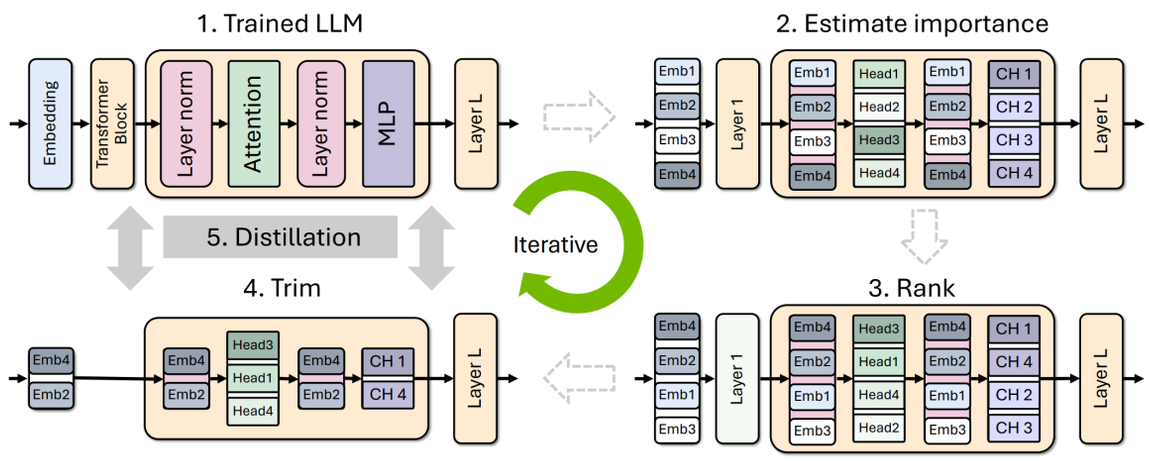
> 核心想法：直接把昂贵模型的一部分拔掉，让它变便宜

...and then fix it up.

> ……然后再把它修好。

Paper from NVIDIA [Muralidharan+ 2024]

> NVIDIA 的论文 [Muralidharan+ 2024]



Algorithm:

> 算法：

1. Identify important {layer, head, hidden dimension} on a small calibration dataset (1024 samples)

> 在一个小型校准数据集（1024 个样本）上识别出重要的 {层, 头, 隐藏维度}

2. Remove unimportant layers to get a smaller model

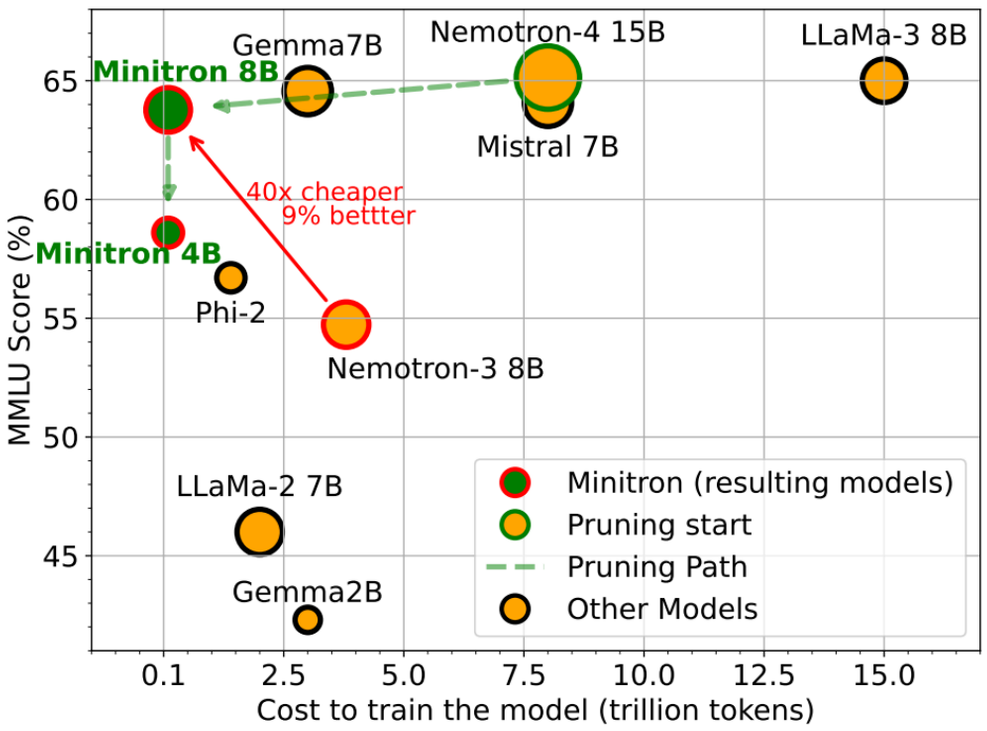
> 移除不重要的层，得到更小的模型

3. Distill the original model into pruned model

> 将原模型蒸馏进剪枝后的模型

Results:

> 结果：



Recall the two stages of inference:

> 回顾推理的两个阶段：

- Prefill: given a sequence, encode tokens in parallel (compute-bound) [note: also gives you probabilities]

> Prefill：给定序列，并行地编码 token（算力受限）[注：同时也会给出概率]

- Generation: generate one token at a time (memory-bound)

> 生成：一次生成一个 token（访存受限）

In other words, checking is faster than generation.

> 换句话说，校验比生成快。

Speculative sampling [Leviathan+ 2022][Chen+ 2023]

> 投机采样（speculative sampling） [Leviathan+ 2022][Chen+ 2023]

- Use a cheaper **draft model** p to guess a few tokens (e.g., 4)

> 用一个更便宜的草稿模型 p 去猜几个 token（例如 4 个）

- Evaluate with target model q (process tokens in parallel), and accept if it looks good

> 用目标模型 q 评估（并行处理这些 token），看着没问题就接受

[Speculative sampling video]

> [投机采样视频]

[article]

Algorithm 2 Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead K and minimum target sequence length T.
Given auto-regressive target model $q(.|.)$, and auto-regressive draft model $p(.|.)$, initial prompt sequence $x_0, \ldots, x_t$.
Initialise $n \leftarrow t$.
while $n < T$ **do**
  for $t = 1 : K$ **do**
    Sample draft auto-regressively $\tilde{x}_t \sim p(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_{t-1})$
  end for
  In parallel, compute $K + 1$ sets of logits from drafts $\tilde{x}_1, \ldots, \tilde{x}_K$:

$$q(x|, x_1, \ldots, x_n),\ q(x|, x_1, \ldots, x_n, \tilde{x}_1),\ \ldots,\ q(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_K)$$

  for $t = 1 : K$ **do**
    Sample $r \sim U[0, 1]$ from a uniform distribution.
    if $r < \min\left(1, \frac{q(x|x_1,\ldots,x_{n+t-1})}{p(x|x_1,\ldots,x_{n+t-1})}\right)$, **then**
      Set $x_{n+t} \leftarrow \tilde{x}_t$ and $n \leftarrow n + 1$.
    else
      sample $x_{n+t} \sim (q(x|x_1, \ldots, x_{n+t-1}) - p(x|x_1, \ldots, x_{n+t-1}))_+$ and exit for loop.
    end if
  end for
  If all tokens $x_{n+1}, \ldots, x_{n+K}$ are accepted, sample extra token $x_{n+K+1} \sim q(x|, x_1, \ldots, x_n, x_{n+K})$ and set $n \leftarrow n + 1$.
end while

This is modified rejection sampling with proposal p and target q

> 这是改进版的拒绝采样：提议分布为 p，目标分布为 q

Modification: always generate at least one candidate (rejection sampling will keep looping)

> 改动之处：始终至少生成一个候选（拒绝采样可能会一直循环下去）

Key property: guaranteed to be an **exact sample** from the target model!

> 关键性质：保证得到目标模型的精确采样！

Proof by example: assume two vocabulary elements {A, B}

> 用例子来证明：假设词表里有两个元素 {A, B}

- Target model probabilities: [q(A), q(B)]

> 目标模型的概率：[q(A), q(B)]

- Draft model probabilities: [p(A), p(B)]

> 草稿模型的概率：[p(A), p(B)]

- Assume p(A) > q(A) [draft model oversamples A].

> 假设 p(A) > q(A) [草稿模型对 A 过采样]。

- Therefore p(B) < q(B) [draft model undersamples B].

> 因此 p(B) < q(B) [草稿模型对 B 欠采样]。

- Residual probabilities max(q-p, 0): [0, 1]

> 残差概率 max(q-p, 0)：[0, 1]

Compute the probabilities of speculatively sampling a token:

> 计算投机采样出某个 token 的概率：

- P[sampling A] = p(A) * (q(A) / p(A)) + p(B) * 1 * 0 = q(A)

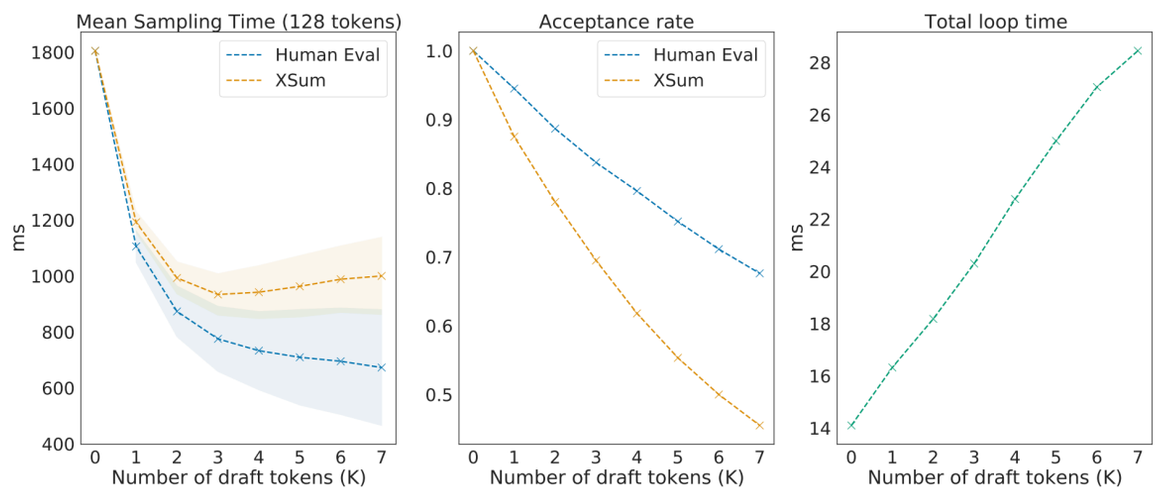
> P[采样到 A] = p(A) * (q(A) / p(A)) + p(B) * 1 * 0 = q(A)

- P[sampling B] = p(B) * 1 + p(A) * (1 - q(A) / p(A)) * 1 = q(B)

> P[采样到 B] = p(B) * 1 + p(A) * (1 - q(A) / p(A)) * 1 = q(B)

Table 1 | **Chinchilla performance and speed on XSum and HumanEval with naive and speculative sampling at batch size 1 and** $K = 4$. XSum was executed with nucleus parameter $p = 0.8$, and HumanEval with $p = 0.95$ and temperature 0.8.

Sampling Method	Benchmark	Result	Mean Token Time	Speed Up
ArS (Nucleus)	XSum (ROUGE-2)	0.112	14.1ms/Token	1×
SpS (Nucleus)		0.114	7.52ms/Token	1.92×
ArS (Greedy)	XSum (ROUGE-2)	0.157	14.1ms/Token	1×
SpS (Greedy)		0.156	7.00ms/Token	2.01×
ArS (Nucleus)	HumanEval (100 Shot)	45.1%	14.1ms/Token	1×
SpS (Nucleus)		47.0%	5.73ms/Token	2.46×



In practice:

> 实践中：

- Target model has 70B parameters, draft model has 8B parameters

> 目标模型有 70B 参数，草稿模型有 8B 参数

- Target model has 8B parameters, draft model has 1B parameters

> 目标模型有 8B 参数，草稿模型有 1B 参数

- Try to make draft model as close to target (distillation)

> 尽量让草稿模型接近目标模型（蒸馏）

Extensions to improve the draft model:

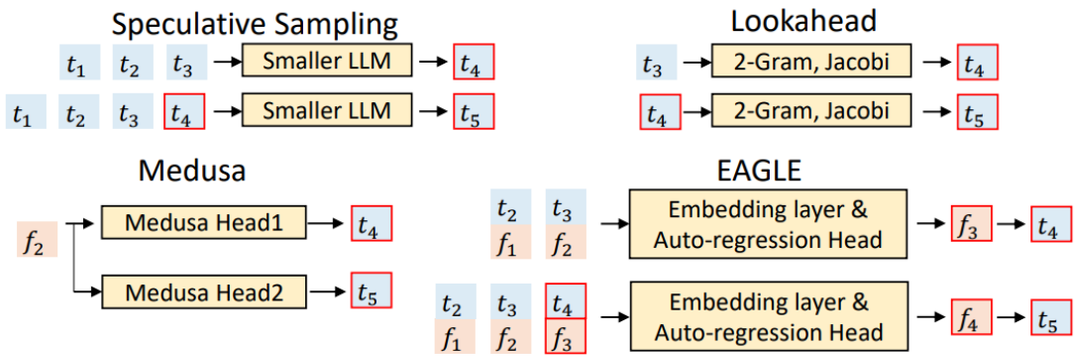
> 改进草稿模型的扩展：

- Medusa: draft model generates multiple tokens in parallel [Cai+ 2024]

> Medusa：草稿模型并行生成多个 token [Cai+ 2024]

- EAGLE: draft model takes high-level features from target model [Li+ 2024]

> EAGLE：草稿模型从目标模型获取高层特征 [Li+ 2024]



Summary:

> 总结：

- Exact sampling from target model (thanks to math)!

> 从目标模型精确采样（多亏了数学）！

- Exploits asymmetry between checking and generation

> 利用了校验与生成之间的不对称性

- Lots of room for innovation on the draft model (involves training)

> 草稿模型上还有大量创新空间（涉及训练）

[Orca: A Distributed Serving System for Transformer-Based Generative Models][talk]
Problem:

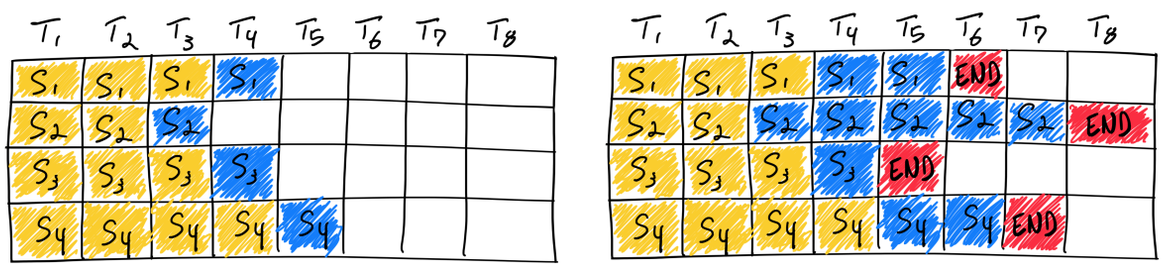
> 问题：

- Training: get a dense block of tokens (batch size x sequence length)

> 训练：拿到的是一整块稠密的 token（batch size x 序列长度）

- Inference: requests arrive and finish at different times, so you have a ragged array

> 推理：请求到达和结束的时间各不相同，所以是一个参差不齐的数组（ragged array）



Solution: iteration-level scheduling

> 解法：迭代级调度（iteration-level scheduling）

- Decode step by step

> 一步一步地 decode

- Add new requests to the batch as they arrive (so don't have to wait until generation completes)

> 新请求一到达就加入 batch（这样不必等到整批生成结束）

Problem:

> 问题：

- Batching only works when all sequences have the same dimensionality (right?)

> 批处理只有在所有序列维度相同时才行得通（对吧？）

- But each request might have a different length

> 但每个请求的长度可能不同

Solution: selective batching

> 解法：选择性批处理（selective batching）

- Training: when all sequences of the same length, operate on a B x S x H tensor

> 训练：所有序列等长时，在 B x S x H 的张量上操作

- But we might have different lengths: [3, H], [9, H], [5, H], etc.

> 但长度可能不同：[3, H]、[9, H]、[5, H] 等

- Attention computation: process each sequence separately

> 注意力计算：每条序列单独处理

- Non-attention computation: concatenate all the sequences together to [3 + 9 + 5, H]

> 非注意力计算：把所有序列拼接到一起，变成 [3 + 9 + 5, H]

Paper that introduced vLLM in addition to PagedAttention [Kwon+ 2023]

> 除了 PagedAttention 之外还提出了 vLLM 的论文 [Kwon+ 2023]

Previous status quo:

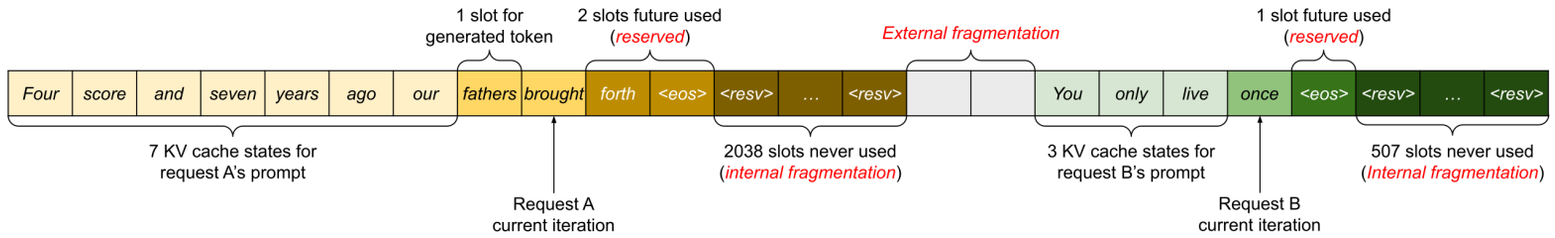
> 此前的常规做法：

- Request comes in

> 请求到来

- Allocate section of KV cache for prompt and response (up to a max length)

> 为提示词和回复分配一段 KV cache（按最大长度预留）



Problem: fragmentation (what happens to your hard drive)

> 问题：碎片化（就是你的硬盘会遇到的那种事）

- But this is wasteful since we might generate much fewer tokens (internal fragmentation)!

> 但这很浪费，因为实际生成的 token 可能少得多（内部碎片）！

- Might be extra unused space between sections (external fragmentation)!

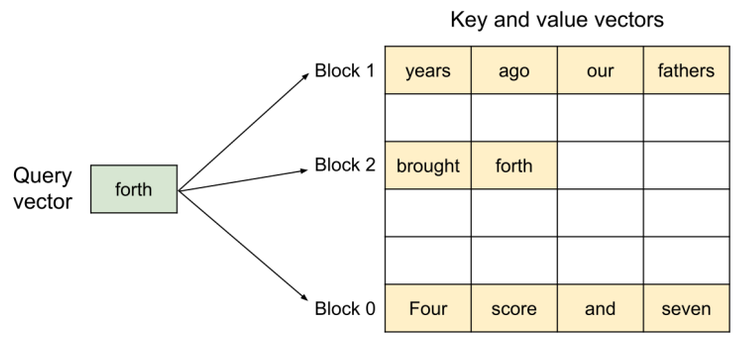
> 各段之间还可能留有多余的空闲空间（外部碎片）！

Solution: PagedAttention (remember operating systems)

> 解法：PagedAttention（想想操作系统）

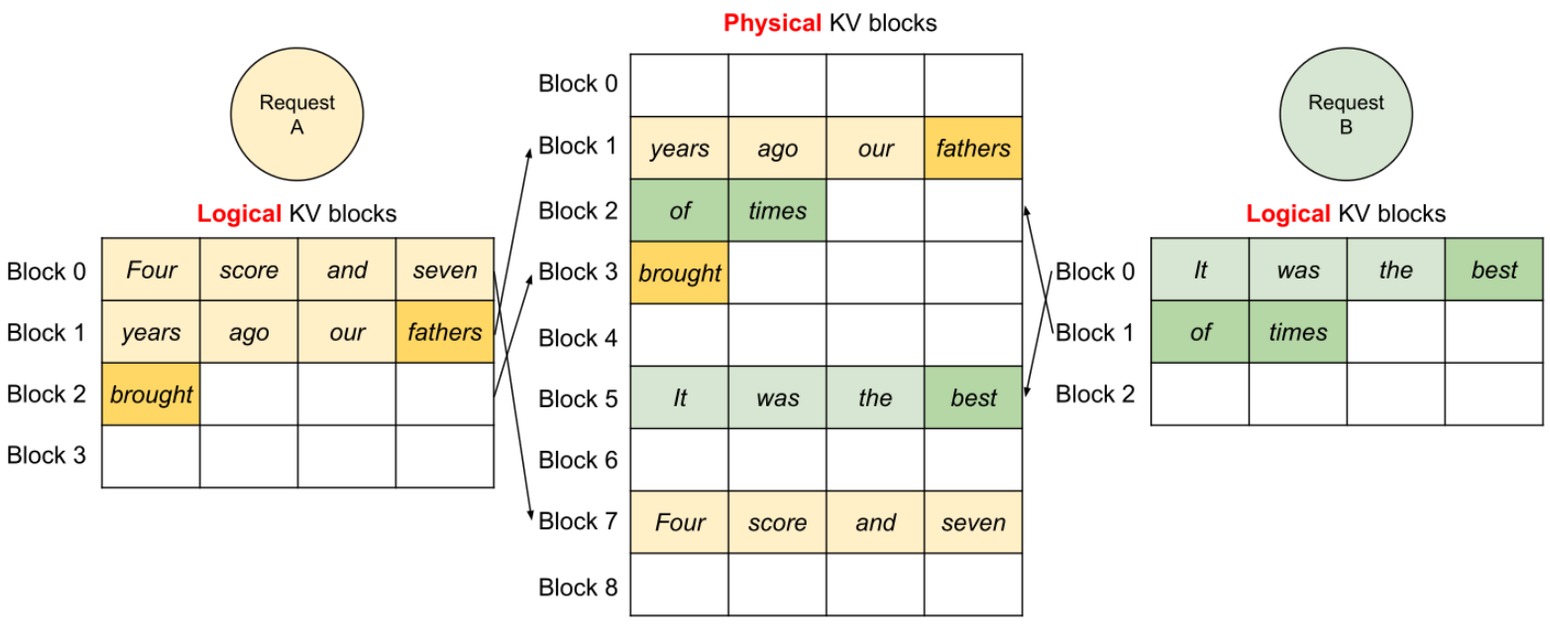
- Divide the KV cache of a sequence into non-contiguous **blocks**

> 把一条序列的 KV cache 划分成不连续的块



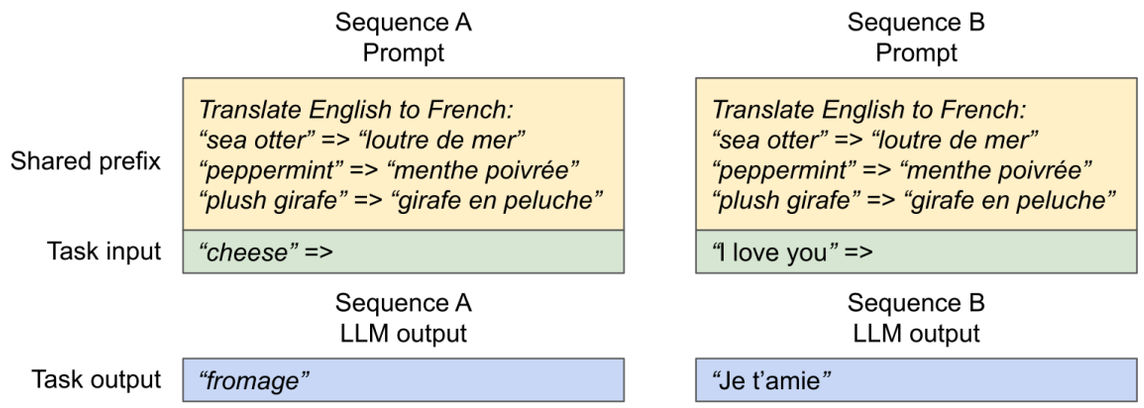
Two requests share the KV caches:

> 两个请求共享 KV cache：



In general, multiple types of sharing KV caches across sequences:

> 一般地，跨序列共享 KV cache 有多种类型：



- Sharing the system prompt

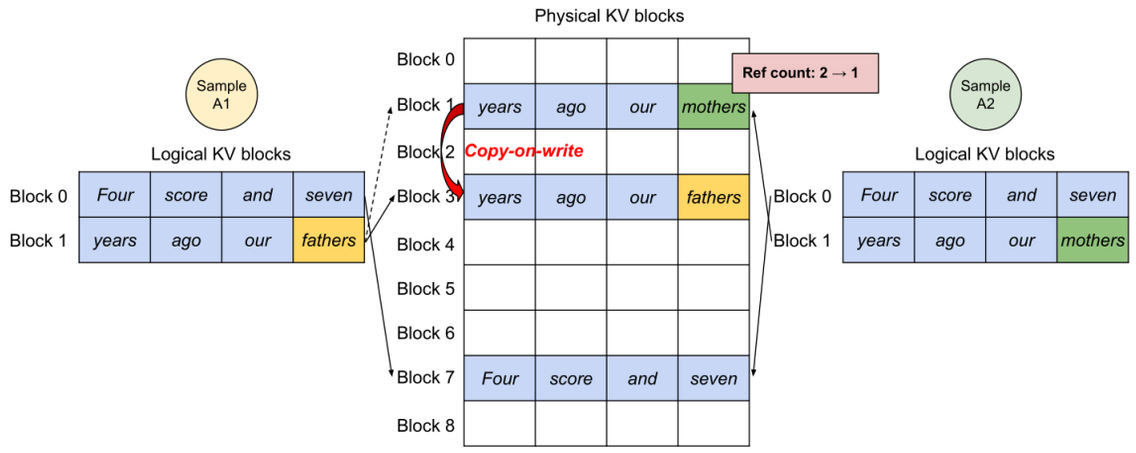
> 共享系统提示词

- Sampling multiple responses per prompt (e.g., for program synthesis)

> 每个提示词采样多个回复（例如用于程序合成）

Solution: share prefixes, copy-on-write at the block level

> 解法：共享前缀，在块级做写时复制（copy-on-write）



Other vLLM optimizations:

> vLLM 的其他优化：

- Kernel to fuse block read and attention (reduce kernel launch overhead)

> 融合块读取与注意力的 kernel（减少 kernel 启动开销）

- Use latest kernels (FlashAttention, FlashDecoding)

> 使用最新的 kernel（FlashAttention、FlashDecoding）

- Use CUDA graphs to avoid kernel launch overhead

> 使用 CUDA graph 避免 kernel 启动开销

Summary: use ideas from operating systems (paging) to make use of memory for dynamic workloads

> 总结：借用操作系统的思想（分页）为动态负载用好内存